Sorting Algorithms — Performance Study

S1.02 — Algorithm Demonstration

n25027444

March 25, 2026

Contents

1 Introduction **2**

2 The Algorithms **2**
2.1 Bubble Sort 2
2.2 Selection Sort 2
2.3 Insertion Sort 2
2.4 Quicksort (Fixed Pivot) 3
2.5 Quicksort (Random Pivot) 3
2.6 std::sort (C++ Standard Library) 3

3 Results and Plots **3**
3.1 Execution Time 4
3.2 Number of Comparisons 7

4 Comparison and Analysis **9**
4.1 Slow vs Fast: the O(n^2) wall 9
4.2 The impact of the input shape 10
4.3 Fixed pivot vs random pivot Quicksort 10
4.4 How close are we to std::sort? 10

5 Conclusion **10**

Introduction

In this project, the goal is to study how different sorting algorithms behave depending on the size and the shape of the input array. Sorting is one of the most fundamental problems in computer science — almost every program needs to organize data at some point, so it is important to understand which algorithm is the most efficient and in which situation.

We tested six algorithms: Bubble Sort, Insertion Sort, Selection Sort, Quicksort (with a fixed pivot), Quicksort (with a random pivot), and the standard `std::sort` from the C++ library. For each one, we measured two things: the execution time in microseconds, and the number of comparisons performed. We ran them on arrays of sizes from 1 000 to 100 000 elements, with three different data patterns:

- **Type 1** — fully random array
- **Type 2** — first half sorted, second half random
- **Type 3** — first half reversed, second half random

The code is written in C++ and compiled with `-Ofast` to get realistic timings. All results are stored in a CSV file and plotted with Python/matplotlib.

The Algorithms

Bubble Sort

Bubble sort is probably the most well-known sorting algorithm, and also one of the least efficient. The idea is very simple: we go through the array from left to right, and every time we find two adjacent elements that are in the wrong order, we swap them. We repeat this process until no swap happens in a full pass, which means the array is sorted.

The name comes from the fact that large elements "bubble up" to the right after each pass. We added a small optimization: if a full pass produces zero swaps, we stop early — the array is already sorted. This helps a lot on nearly sorted arrays, but in the worst case (reversed array), the algorithm still has to do roughly $n^2/2$ comparisons, which makes it $\mathbf{O}(n^2)$.

Selection Sort

Selection sort works in a different way. It splits the array into two parts: a sorted part on the left, and an unsorted part on the right. At each step, we scan the unsorted part to find the smallest element, and we move it to the end of the sorted part (by swapping).

This algorithm always does exactly $\frac{n(n-1)}{2}$ comparisons, no matter what the input looks like. This is both a strength (predictable) and a weakness (no early exit possible). It is also $\mathbf{O}(n^2)$, and in practice slower than insertion sort on partially sorted data.

Insertion Sort

Insertion sort builds the sorted array one element at a time. We take each element and insert it into the correct position among the elements already sorted. To do this, we shift elements to the right one by one until we find the right spot.

This algorithm is very efficient when the array is already mostly sorted, because each element barely moves. In the best case (sorted input) it only needs $n - 1$ comparisons, making it $\mathbf{O}(n)$. In the worst case (reversed input) it degrades to $\mathbf{O}(n^2)$, same as the others, but it is generally faster in practice for small or partially sorted arrays.

Quicksort (Fixed Pivot)

Quicksort is a much more efficient algorithm based on a strategy called divide and conquer. We pick one element called the *pivot* (here, always the first element of the array), and we split the array into three groups: elements smaller than the pivot, elements equal to the pivot, and elements larger than the pivot. Then we recursively sort the first and the third groups.

On average, this gives $\mathbf{O}(n \log n)$ comparisons, which is much better than the quadratic algorithms. However, if the pivot is always the worst possible choice — for example, always the smallest or largest element — the algorithm degrades to $\mathrm{O}(n^2)$. This can happen on already sorted or reversed arrays when we always pick the first element.

Quicksort (Random Pivot)

This version is identical to the previous one, except the pivot is chosen randomly. This small change makes the worst case extremely unlikely in practice, because the bad inputs for Quicksort with fixed pivot (sorted or reversed arrays) become just average cases here. The expected complexity stays $\mathbf{O}(n \log n)$, and it is more robust than the fixed pivot version.

std::sort (C++ Standard Library)

The last algorithm is not one we implemented ourselves — it is the `std::sort` function from the C++ standard library. Under the hood, it uses a hybrid algorithm called *introsort*, which combines Quicksort, Heapsort, and Insertion Sort. It switches between them depending on the situation to always have $\mathbf{O}(n \log n)$ performance, even in the worst case. We included it as a reference point to see how close our implementations get to a professionally optimized sort.

Results and Plots

We separated the algorithms into two groups for readability: the "slow" group (Bubble, Selection, Insertion) and the "fast" group (Quicksort, Quicksort random, std::sort).

Execution Time

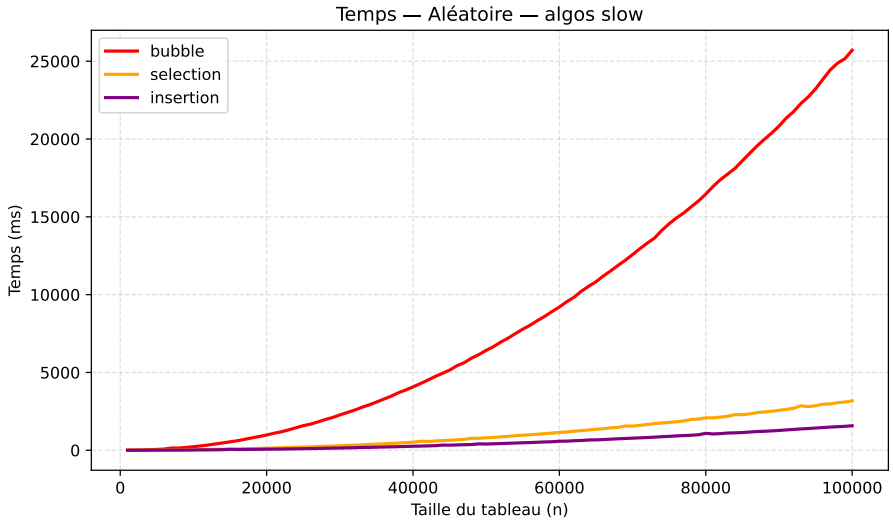

Figure 1: Execution time — slow algorithms — random array (type 1)

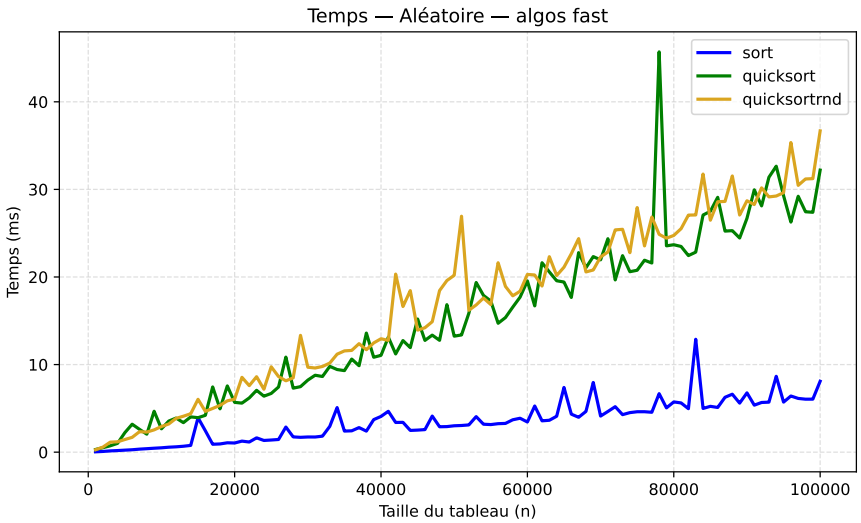

Figure 2: Execution time — fast algorithms — random array (type 1)

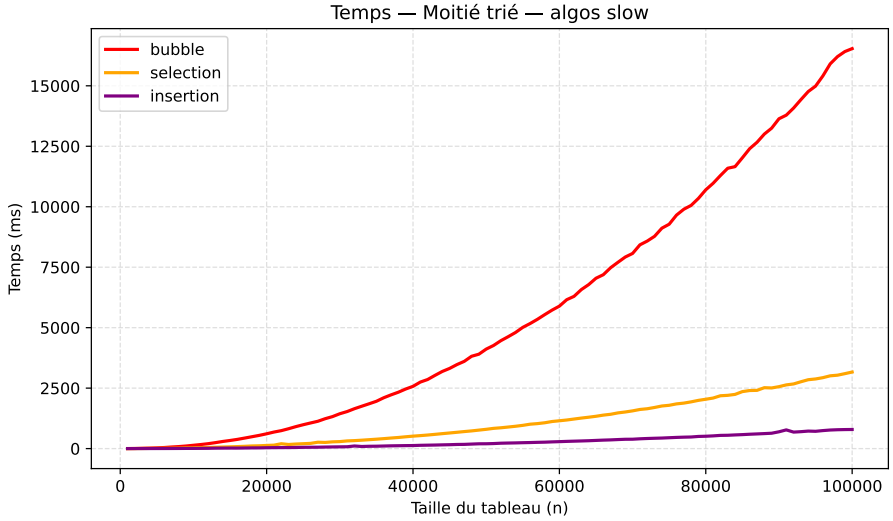

Figure 3: Execution time — slow algorithms — half sorted array (type 2)

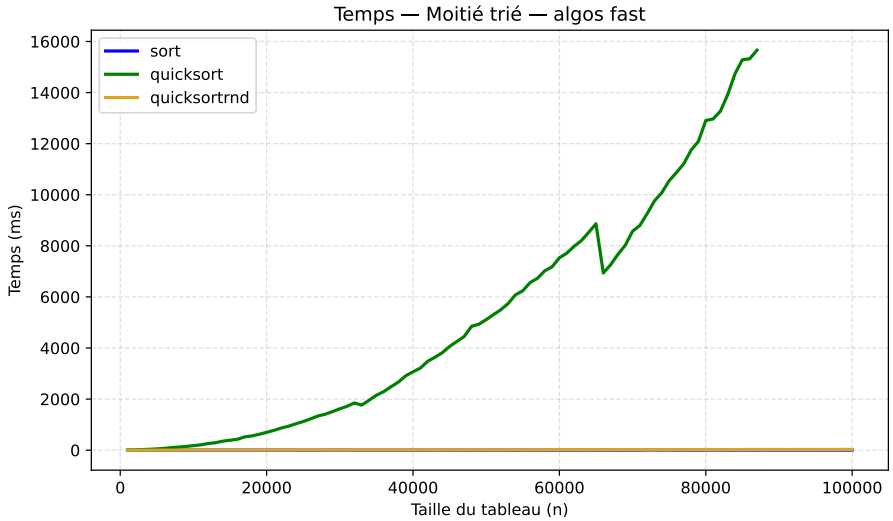

Figure 4: Execution time — fast algorithms — half sorted array (type 2)

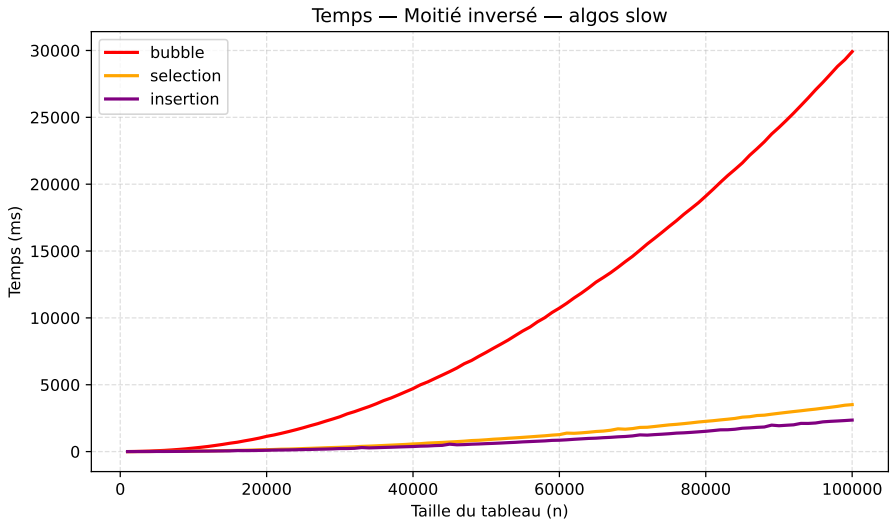

Figure 5: Execution time — slow algorithms — half reversed array (type 3)

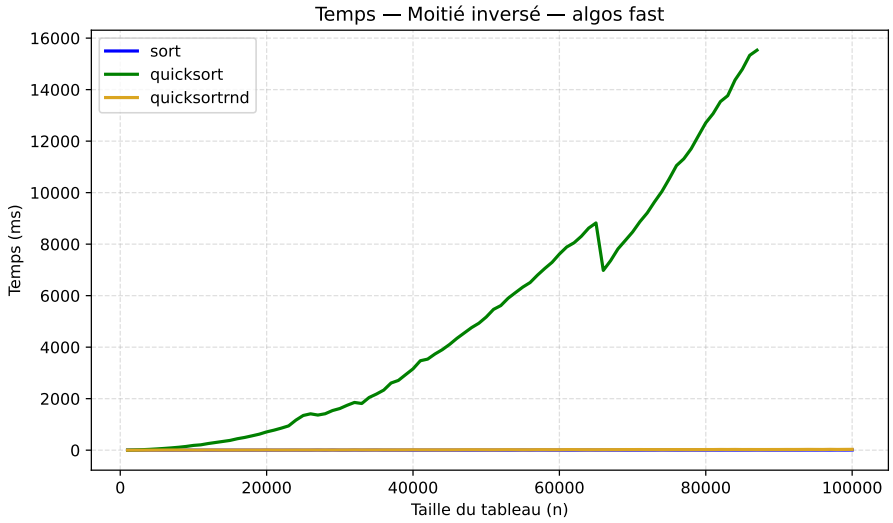

Figure 6: Execution time — fast algorithms — half reversed array (type 3)

Number of Comparisons

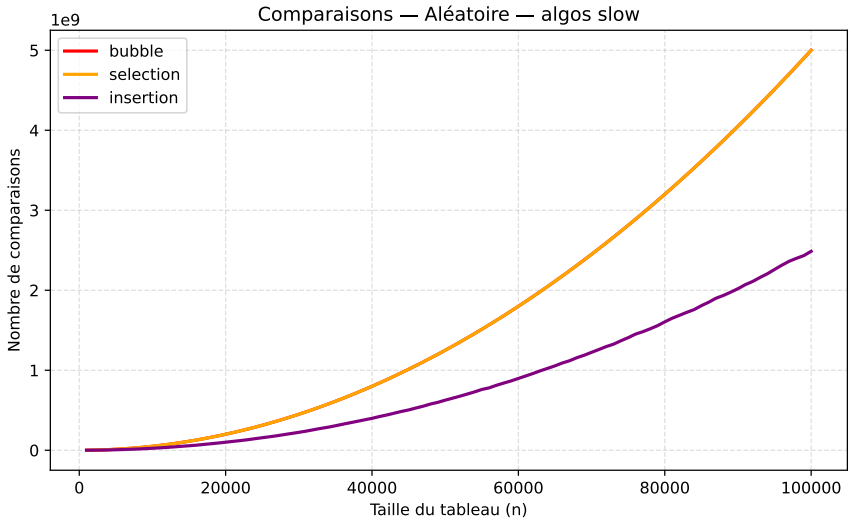

Figure 7: Number of comparisons — slow algorithms — random array (type 1)

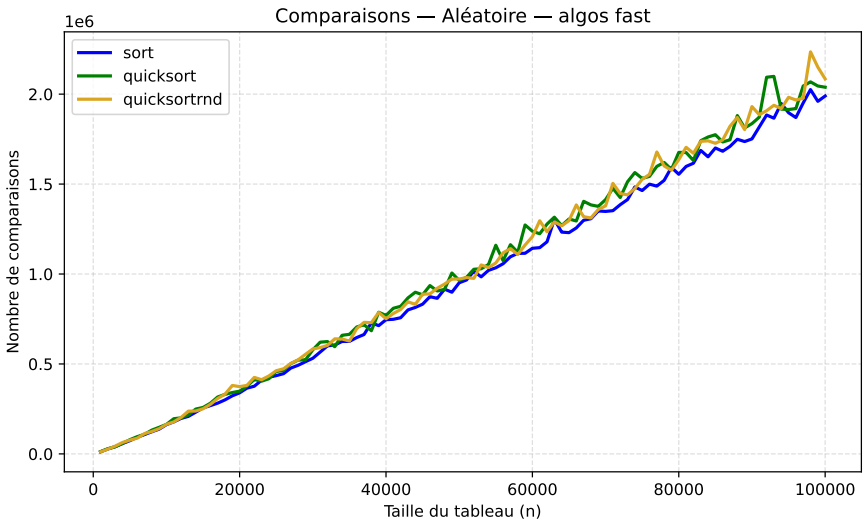

Figure 8: Number of comparisons — fast algorithms — random array (type 1)

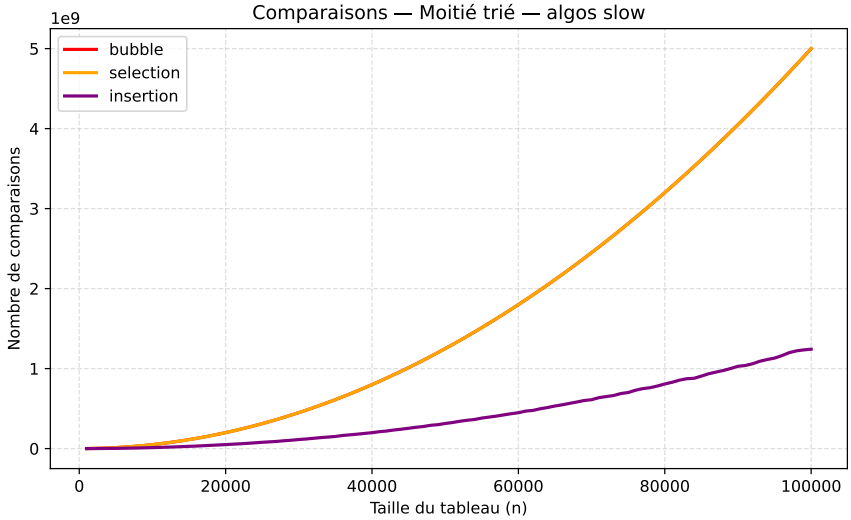

Figure 9: Number of comparisons — slow algorithms — half sorted array (type 2)

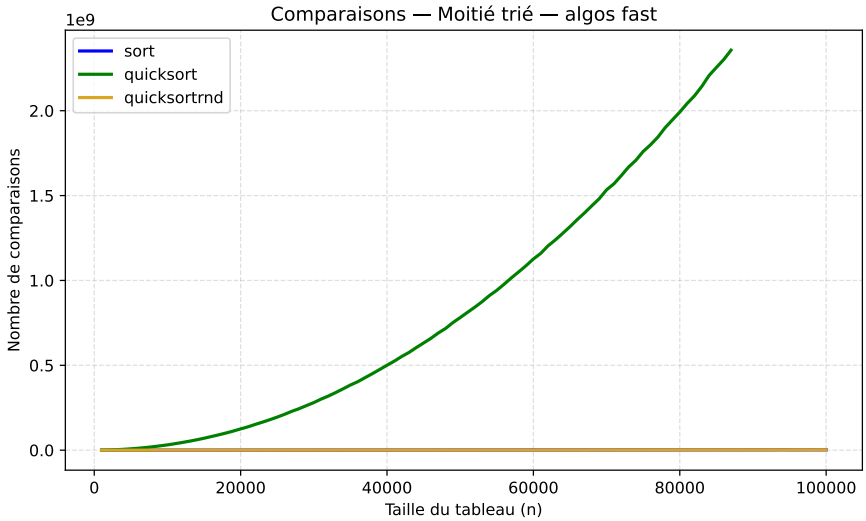

Figure 10: Number of comparisons — fast algorithms — half sorted array (type 2)

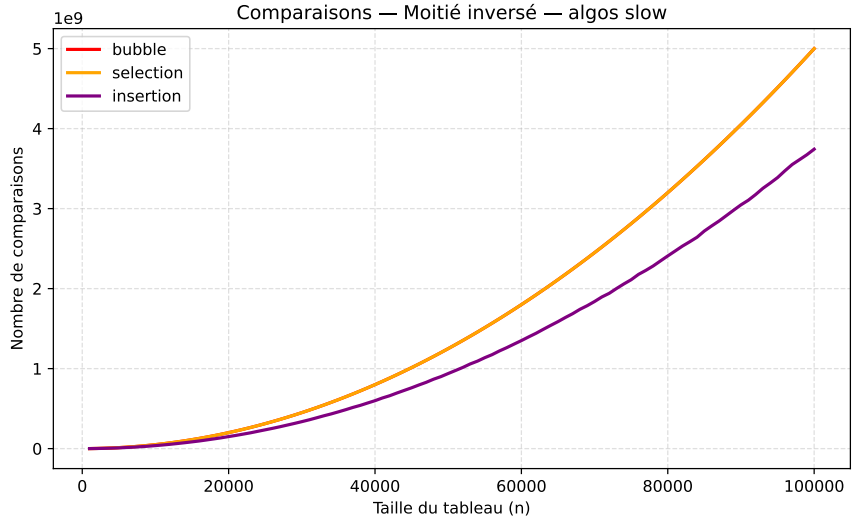

Figure 11: Number of comparisons — slow algorithms — half reversed array (type 3)

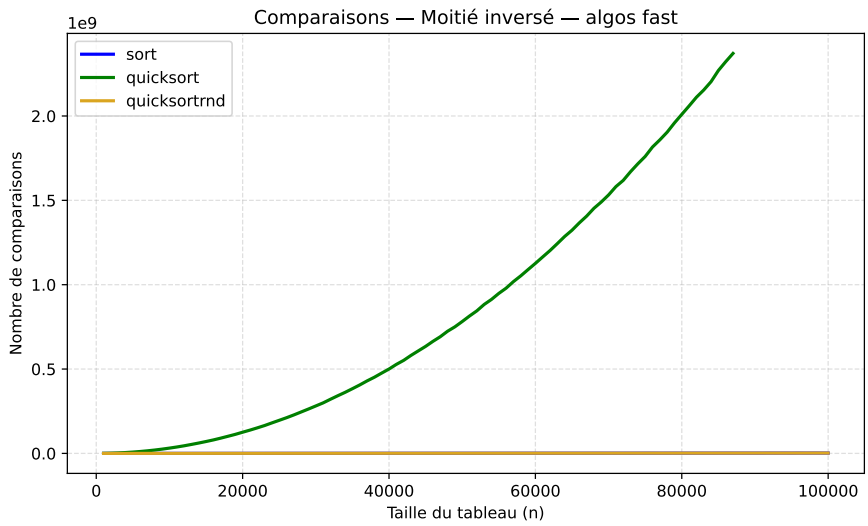

Figure 12: Number of comparisons — fast algorithms — half reversed array (type 3)

Comparison and Analysis

Slow vs Fast: the O(n^2) wall

The most obvious thing we can see from the plots is the huge difference between the two groups. Bubble, Selection, and Insertion sort all have curves that grow very fast — their time roughly quadruples every time the array size doubles. This is the signature of O(n^2) algorithms. At $n = 60{,}000$ elements, bubble sort already takes several seconds, while std::sort handles $n = 100{,}000$ in under a millisecond.

In terms of comparisons, Selection Sort is perfectly predictable: it always does exactly $\frac{n(n-1)}{2}$ comparisons regardless of the input. Bubble and Insertion can do fewer on easier inputs, which is visible in the type 2 plots where the input is partially sorted.

The impact of the input shape

The type of input has a big effect on the slow algorithms, but almost none on the fast ones.

- **Insertion Sort** benefits a lot from partially sorted data (type 2). When elements are already roughly in order, each insertion only needs a few shifts. On type 3 (half reversed), it is at its worst for the first half.
- **Bubble Sort** also benefits from its early-stop optimization on type 2 data.
- **Selection Sort** is completely unaffected — it always scans the full unsorted portion.
- **Quicksort and std::sort** show almost no difference between the three types. Their curves are nearly identical, which shows they are robust to the input shape.

Fixed pivot vs random pivot Quicksort

In theory, Quicksort with a fixed pivot (always the first element) can be much worse on sorted or nearly sorted data. However, in our tests, both versions perform similarly across all three types. This is probably because our type 2 and type 3 arrays are not fully sorted — only half of the elements are, which limits the worst-case effect.

How close are we to std::sort?

Our random pivot Quicksort is actually quite close to `std::sort` in terms of comparison count. The time difference is larger because `std::sort` uses additional tricks — it avoids allocating new vectors at each recursive call (unlike our implementation which creates `v1`, `v2`, `v3` every time), and it switches to Insertion Sort for small sub-arrays. These memory and cache optimizations make a real difference at large sizes.

Conclusion

This study confirms what complexity theory predicts. The $O(n^2)$ algorithms are easy to understand and implement, but they simply cannot keep up with larger inputs. Above 20 000–30 000 elements, they become too slow to be useful in any real application.

The $O(n \log n)$ algorithms, especially std::sort and random pivot Quicksort, handle even 100 000 elements almost instantly. The choice of pivot in Quicksort matters mostly on very specific inputs — for general use, a random pivot is safer.

If I had to pick one algorithm for everyday use, I would always go with `std::sort`: it is fast, safe, and already written correctly. But studying the simpler algorithms helped me understand *why* it is better, which is the real point of this project.